Exception Handling

CS 22B, Spring 2026

Module 04: Exception Handling

Presented by Jessica Huynh-Westfall, Ph.D.

02

AGENDA

- Exception handling
  - Syntax Errors
  - Runtime Errors

03

Chapter 14: Exceptions

Quick Python

Class exercise

04

Scenario:

You are working on a file in your word processor. You want to write files to your disk, however you may run out of disk space before all of the data is written.

How do you handle this problem?

Types of Errors

05

Errors occur when the Python interpreter encounters an issue during execution.

Types of error are

- **Syntax Errors**: inconsistent code structure
- **Runtime Errors**: issues occurring during program execution
  - wrong data type
  - wrong function
  - wrong variable name

Example of Syntax Error

07

Syntax errors are the most common type of error

- Parsing Errors
  - Occurs when parser detects incorrect statement.
  - Parser points out the line with the error
- Implementations execute instructions directly without earlier compiling a program

```
>>> while True print("Hello World!")
  File "<python-input-0>", line 1
    while True print("Hello World!")
               ^^^^^
SyntaxError: invalid syntax
```

Example of Runtime Error

08

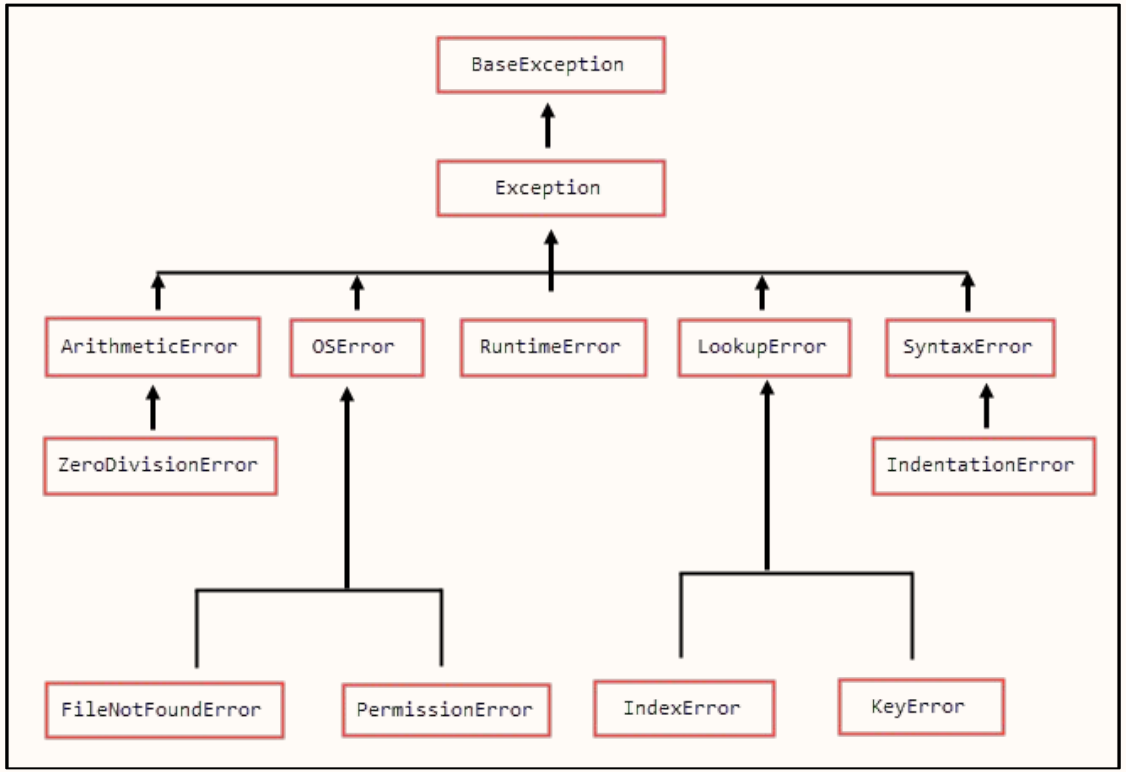
RuntimeError are built-in exception raised when an error occurs during program execution

Common RuntimeError

- ZeroDivisionError - Division by Zero
- NameError - Using undefined variable or function
- FileNotFoundError - Accessing a file that does not exist
- TypeError - Performing an operation on incompatible types
- IndexError - Accessing a non-existent list element or dictionary key

```
>>> print(x)
           ^
NameError: name 'x' is not defined
```

Type of Exceptions



List of exceptions

How do you handle errors?

Errors occur when the Python interpreter encounters an issue during execution.

- Identify Error message - Python will specifies exception type
- Code logic - Review code for potential logic, mathematical, or typographical errors
- Validate data type - Confirm usage of appropriate data types and values passed to function or operations
- **Exception Handling** - Employ try, except, else, and finally block to manage expected runtime issues to prevent program from crashing.

Exception handling

Exception handling in Python is used to manage errors that occur during program execution. This enables more robust code and prevent abrupt crashes.

Involves:

- **try** block - code raise an exception if error
- **except** block - if exception occur in try, code in except block execute
  - Multiple except block can be used to handle different exception
- **else** block - if no exception occur in try, code in else block execute
- **finally** block - block used for cleanup action. Block always executed regardless of if exception occured

Raising an Exception Error

```
>>> print(x)
           ^
NameError: name 'x' is not defined
```

When run, this Raise an Error (Exception)

Handle this error with Exception Handling

```
try :
    print(x)
except:
    print("There is an error")
```

Catch the error (currently all errors)

Raising an Exception Error


```
>>> print(x)
           ^
NameError: name 'x' is not defined
```

When run, this Raise an Error (Exception)

Handle this error with Exception Handling

```
try :
    print(x)
except NameError:
    print("NameError; undefined variable or function")
```

Specify the type of error to catch

Try and Except Statement

Catching exceptions with **try and except** statements

- **try** clause: Statements that can cause exceptions are kept inside the cause
- **except** clause: Statements that handle the exception are written inside the cause

Try and Except block

Used to catch and handle exceptions. Try statement executes code as a normal part of the program. The code that follows the except statement is the program's response to any exceptions in the preceding try clause:

```
try:
    {  Code to test if there will be an exception
except:
    {  Execute code when there is an exception
```

Multiple Except blocks can be implemented


```
x=10
try:
    print(x/0)
except NameError:
    print("NameError; undefined variable or function")
except ZeroDivisionError:
    print("Cannot divide by Zero")
```

What if there are no errors?

Try, Except and Else

try:

{ Code

except:

{ Execute code when there's an exception

else:

{ No exception. Run this code block.

Multiple Except blocks can be implemented

```
x=10
try:
    print(x/0)
except NameError:
    print("NameError; undefined variable or function")
except ZeroDivisionError:
    print("Cannot divide by Zero")
else:
    print("No errors. Yay!")
```

Oh no! This will raise an error

error is raised, so else never reached

Try, Except, Else and Finally

try:

{ Code

except:

{ Execute code when there's an exception

else:

{ No exception. Run this code block.

finally:

{ Always run this code

Raising your own error


```
y=10
try:
    if not type(y) is str:
        raise TypeError("y should be a string")
except NameError:
    print("NameError; undefined variable or function")
except ZeroDivisionError:
    print("Cannot divide by Zero")
except Exception as error:
    print(error)
else:
    print("No errors. Yay!")
finally:
    print("This will print in all cases")
```

Raise a built-in error

Defining your own exception

```
class YuckyError(Exception):
    pass
```

Custom exception

```
y=10
try:
    raise YuckyError("This is yucky.")
except NameError:
    print("NameError; undefined variable or function")
except Exception as error:
    print(error)
else:
    print("No errors. Yay!")
```

Passing function into try-except

Signaling that something unexpected or erroneous has occurred

```
def divide_with_even(y,x):
   if x%2 != 0:
      raise Exception("Sorry, no non-even numbers")
```

Error should not propagate any further

```
try:
  divide_with_even(4,3)
except Exception as error:
  print(error)
```

Raising an Exception error

Raised an **exception object** and passed it with custom message. You built the message using an f-string and a self-documenting expression.

Example: Program that expects only numbers up to 5. If unwanted condition occurs, you can raise an error

```
number = 10
if number > 5:
    raise Exception(f"The number should not exceed 5.
({number=})")
print(number)
```

Disable AssertionError

If you make this a num5low.py python script and run, you can disable AssertionError using the **-O command** line option.

- This removes all assert statements.
- Your script runs all the way to the end (ie., print statement) and displayed the number 10

num5low.py

```
number = 1
assert (number < 5), f"The number should not exceed 5.
({number=})"
print(number)
```

```
$ python -O num5low.py
10
```

24

Coding Bug

Coding Bug

The code runs, but produces incorrect results. Does not raise an error.

Example: Using the wrong formula in a calculation

```
dna = "ACTGATCGATTACGTATAGTATTTGCTATCATACATATATATCGATGCGTTCAT"
a_count = dna.count("A")
t_count = dna.count("T")
dna_length = len(dna)
AT_content = a_count + t_count/dna_length *100
AT_content
```

Use AssertionError to debug your program

Here you raise an exception when a certain condition isn't met:

```
number = 1
if number > 5:
    raise Exception(f"The number should not exceed 5.
({number=})")
print(number)
```

Replace conditional statement with assertion to check development

```
number = 1
assert (number < 5), f"The number should not exceed 5.
({number=})"
print(number)
```

Use built-in exceptions (Example Runtime)

Example: Open a file and use a built-in exception.

```
try:
    with open("file.log") as file:
        read_data = file.read()
except:
    print("Couldn't open file.log")
```

If file.log doesn’t exist, then this block of code will output the following:

```
$ python open_file.py
Couldn't open file.log
```

Try, Except and Else

linux_interactions.py

```
def linux_interactions():
    import sys
    if "linux" not in sys.platform:
        raise RuntimeError("Function can only run on Linux systems.")
    print("Doing Linux things.")

try:
        linux_interactions()
except RuntimeError as error:
        print(error)
else:
        try:
                with open("file.log") as file:
                        read_data = file.read()
        except FileNotFoundError as fnf_error:
                print(fnf_error)
```